National University of Computer and Emerging Sciences, Lahore Campus

Department of Artificial Intelligence and Data Science

Generative AI, Fall 2026

Assignment 3: Parameter-Efficient Fine-Tuning for Urdu and Roman Urdu

Instructor: Dr. Maryam Bashir

Instructions

- This is a **group assignment for 3 to 4 students**. Enter your group in the following sheet; each group receives a **group number g**, which fixes your track and seeds (see below).
  https://docs.google.com/spreadsheets/d/1DlvF2jMWDLOw7lcMkM6mnpUEA85Zz5TTCMU662IDEOU/edit?usp=sharing
- Everything runs on the **free Google Colab GPU (T4)**. Members can share the experiments across their Colab accounts.
- The report must include a short **contribution statement** for each member. The **viva is individual**: any member may be asked about any part of the work, and individual marks may be adjusted based on the viva and the contribution statement. Failure to attend the viva results in zero marks.

Objective

In this assignment your group will implement and compare **Full Fine-Tuning, LoRA, QLoRA and IA³** for **Urdu or Roman Urdu** text classification, and present the comparison in a short report. Your comparison should answer these questions:

- **RQ1.** How do the four methods trade off accuracy against trainable parameters, training time and GPU memory for an Urdu or Roman Urdu task?
- **RQ2.** How well do the tokenizers of multilingual models handle Urdu and Roman Urdu, and does this affect accuracy and cost?
- **RQ3.** Which method degrades least when labelled data is scarce?

Tracks, Models and Seeds

Your **track** is ((g − 1) mod 4) + 1. Your **seeds** are g and g + 100. All datasets are on Hugging Face:

Track	Task	Dataset	Size and labels	Script
1	Sentiment analysis	`community-datasets/roman_urdu`	about 20,000 sentences; Positive, Negative, Neutral	Roman Urdu
2	Hate speech detection	`communitydatasets/roman_urdu_hate_speech`	about 10,000 tweets; 2 classes (coarse) and 5 classes (fine)	Roman Urdu

3	Tweet sentiment	`communitydatasets/urdu_sentiment_corpus`	1,000 tweets; Positive, Negative, Objective	Urdu
4	Fake news detection	`communitydatasets/urdu_fake_news`	900 news articles in 5 domains; Real, Fake	Urdu

Models: every group uses two multilingual encoders, `xlm-roberta-base` and `google/muril-basecased` (MuRIL was pre-trained on Indian languages, including Urdu and transliterated text). Track 2 groups report both the coarse and the fine-grained task.

Part 1: Data and Tokenizer Analysis (10%)

- Load your dataset, remove empty rows and exact duplicates, and create a stratified 70/15/15 train, validation and test split (Track 2: use the given splits). Report the class distribution in a table.
- Normalise the text: remove URLs and user mentions, and for Urdu script unify character variants (for example, Arabic and Urdu forms of yeh and kaf). Describe every cleaning step in the report.
- **Tokenizer analysis (RQ2):** for both models, report the **fertility** (average subword tokens per word), the share of texts truncated at your chosen max_length, and the unknown-token rate. Choose max_length from the 95th percentile of token lengths and justify it.

Part 2: Implementation (25%)

Fine-tune **both models** with all four methods, using the Hugging Face Trainer and the PEFT library:

- **Full Fine-Tuning**, as the accuracy and cost baseline.
- **LoRA**, with `task_type="SEQ_CLS"` so that the classification head is also trained.
- **QLoRA**: load the base model in 4-bit NF4 with `BitsAndBytesConfig`, call `prepare_model_for_kbit_training`, then add LoRA. The T4 does not support BF16 well, so use `bnb_4bit_compute_dtype=torch.float16`.
- **IA³** (Infused Adapter by Inhibiting and Amplifying Inner Activations; Liu et al., Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning, NeurIPS 2022): instead of adding new matrices, IA³ learns three vectors per layer that rescale the keys, the values and the feed-forward activations element by element (for example, $K' = l_k \odot K$). The vectors start at 1, so training begins from the pre-trained model; values above 1 amplify a feature and values below 1 inhibit it. It trains only about 0.05% of the parameters and can be merged into the weights after training. Use `IA3Config` in PEFT, and print the model to find the correct module names for each architecture.

Measure in every run: macro-F1 (the main metric, since the classes are imbalanced), accuracy, training time, peak GPU memory using `torch.cuda.max_memory_allocated()`, and the number of trainable parameters.

Part 3: Ablation Studies (35%)

Fixed settings: 3 epochs, effective batch size 16 (use gradient accumulation if needed), learning rate 2×10^{-5} for Full Fine-Tuning and 2×10^{-4} for the PEFT methods, and the same max_length in all runs. Run every experiment with your two seeds and report mean ± standard deviation. Save all configurations and results to a CSV file.

Exp	Method	Factor varied	Setting
01	Full FT	baseline	default settings
02	LoRA	rank r	r = 8, targets query and value, α = 16
03	LoRA	rank r	r = 16, targets query and value, α = 16
04	LoRA	placement	r = 16, attention only vs attention + feedforward
05	QLoRA	quantization	NF4, double quantization on, r = 16, paged AdamW
06	QLoRA	optimizer	as 05, paged AdamW on vs off
07	IA^3	placement	attention only vs attention + feed-forward
08	Best of each	data size (RQ3)	Tracks 1 and 2: 10%, 30%, 100%; Tracks 3 and 4: 25%, 50%, 100%

Required plots: LoRA rank against macro-F1; Pareto plots of macro-F1 against peak memory and against training time for all configurations; macro-F1 against training data size for each method; and fertility against macro-F1 for the two models.

Part 4: Comparison Report (30%)

Submit a short report (PDF, at most 6 pages) that presents your comparison with tables and plots:

- **Setup:** dataset, cleaning steps, splits, max_length and all hyperparameters, in one table.
- **Tokenizer table** from Part 1 (fertility, truncation rate and unknown-token rate for both models).
- **Main results table:** for every experiment and both models, macro-F1, accuracy, training time, peak GPU memory and trainable parameters, as mean ± standard deviation over your two seeds.
- **All required plots** from Part 3, each with a one-line caption saying what it shows.
- **Discussion** (about one page): answer RQ1 to RQ3 using your numbers. Which configurations give the best trade-off between accuracy and cost? Is r = 16 worth its extra cost over r = 8? Does adding the feed-forward layers help? How much memory does QLoRA save, and at what cost in accuracy? Which method is most robust with little data?
- **Contribution statement:** one or two lines on what each member did.

Deliverables

- **Colab notebooks** with all cells executed, so they do not need to be re-run.
- The **results CSV** with every configuration and seed.

- The **comparison report** (PDF).

Marks Distribution

Part	Task	Marks
1	Data and tokenizer analysis	5
2	Implementation of four methods on two models	15
3	Ablation studies and plots	15
4	Comparison report	10
Total		45